Database Type Overview

Name: **Amazon Aurora**
Category: Cloud-Native Relational Database (RDBMS) – Managed Distributed SQL System
Representative Systems: Amazon Aurora (MySQL-compatible and PostgreSQL-compatible editions)

Core Concepts and Features

Data Model

Aurora -> relational model. Data organized in tables composed of rows and columns => relational constraints (primary key, foreign key) and joins. Fully compatible with MySQL and PostgreSQL wire protocols.

Storage Model

A distributed, log-structured storage system that separates compute (database engine) from storage.

- Data is stored in 10 GB segments (so as to reduce Mean Time To Repair (MTTR)); each replicated six times across three Availability Zones (AZs) (two copies per AZ).
- Writes are performed via log records, making the storage layer append-only and optimized for crash recovery.

Query Language / APIs

Aurora supports:

- SQL (standard relational queries)
- MySQL and PostgreSQL extensions
- JDBC and ODBC APIs for application connectivity
- AWS SDKs and RDS APIs for administrative control

Scalability

Compute scaling: Up to 128 TB storage, with upto 15 read replicas for horizontal read scaling.

Storage scaling: Automatically grows in 10 GB increments without manual provisioning.

Distributed storage allows decoupling of compute and storage

Consistency Model

Strong consistency for reads and writes (ACID compliant). ACID – Atomicity, Consistency, Isolation, Durability
It uses a quorum-based consistency model :- Write quorum: 4/6 replicas must acknowledge a write; Read quorum: 3/6 replicas must respond for a read.
=> Strong consistency while maintaining high availability.

Fault Tolerance and Recovery

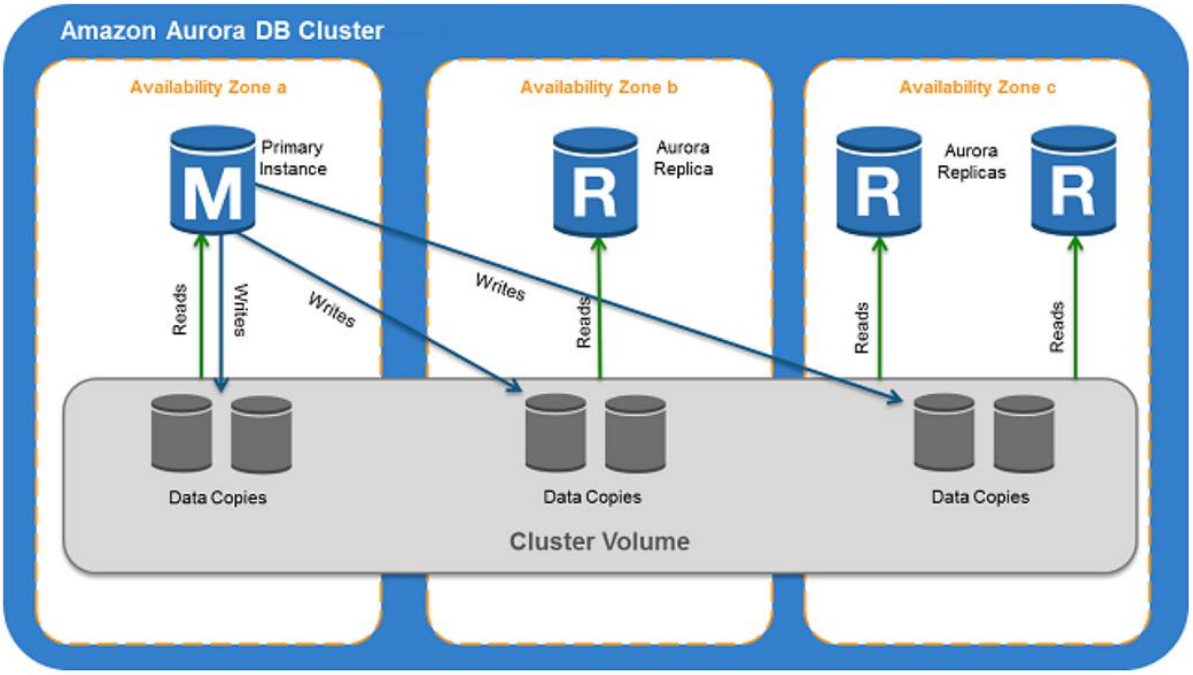
- Survives AZ failures and node crashes without downtime.
- Self-healing storage: corrupted segments are automatically detected and repaired from other replicas.
- Continuous incremental backups to Amazon S3.
- Recovery time is independent of database size (due to log-structured architecture).

Security and Access Control

- Encryption: Data at rest (via KMS – Key Management Service) and in transit (TLS – Transport Layer Security).
- Network security: Integration with VPC and IAM policies.
- Authentication: IAM-based, password-based, or certificate-based user authentication.

AMAZON AURORA ARCHITECTURE

Amazon Aurora DB Cluster Architecture



This figure focuses on Aurora's distributed storage and replication mechanism across multiple Availability Zones (AZs).

Cluster Structure

Aurora's database cluster spans three Availability Zones for high availability.

It includes:

Primary Instance (M): Manages all writes.

Aurora Replicas (R): Handle read requests and can promote to primary on failure.

Cluster Volume

Beneath all instances lies a shared cluster volume replicated six ways across the three AZs (two copies per zone).

The replication happens at the storage layer, not at the database layer, which reduces overhead and latency.

Log-Based Architecture

Instead of writing complete data pages each time, Aurora's storage nodes process database log records directly. They are ordered using Log Sequence Numbers (LSN) (so that nodes can rebuild data pages independently when required).

Quorum System

Writes => Aurora waits for acknowledgment from 4 out of 6 copies.

Reads => 3 out of 6 copies.

Ensures data consistency even if one or more nodes or AZs fail.

Performance Benefits

By moving log processing to the storage tier, Aurora eliminates the double-write penalty of traditional databases.

Parallel log processing and read-ahead caching improve throughput, making Aurora upto 5x faster than standard MySQL and upto 3x faster than PostgreSQL

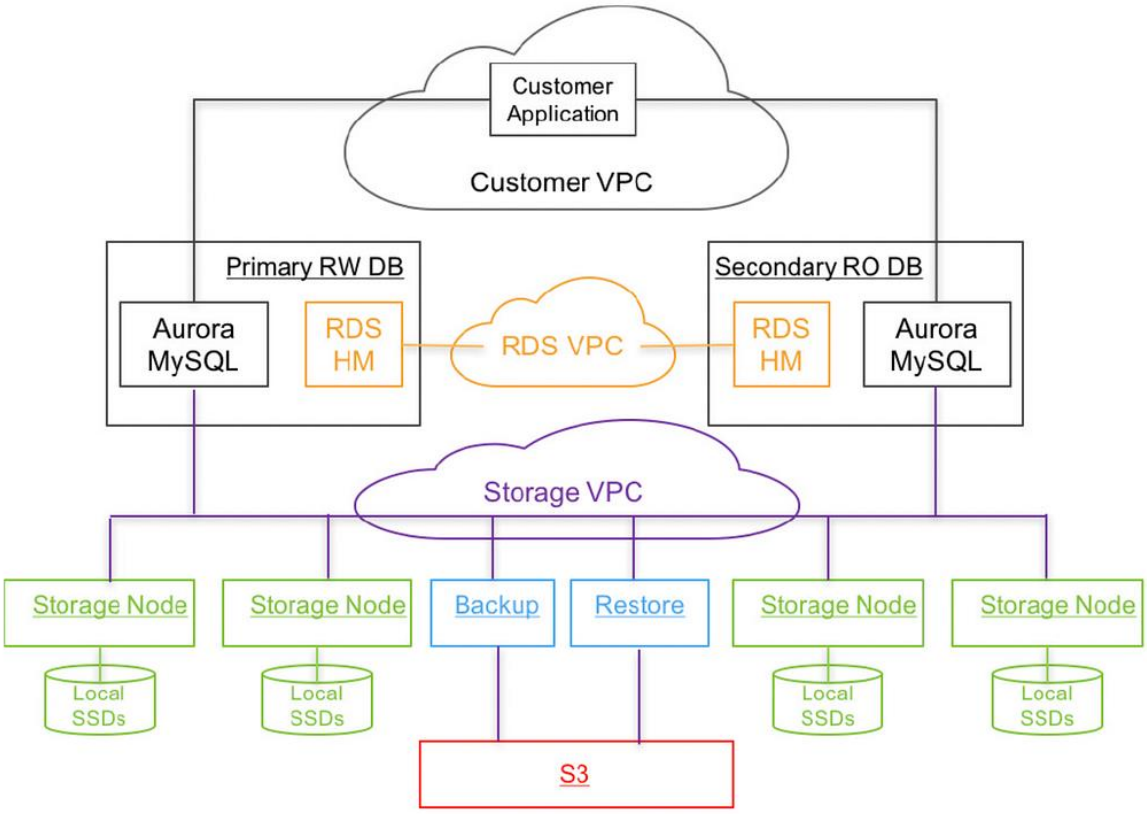
Failure Recovery and Scaling

If a compute instance fails, a new one can attach to the existing cluster volume instantly - no data copying or crash recovery required.

Read replicas can be added or removed seamlessly (up to 15).

Aurora's design enables sub-second failover and near-instant scaling.

Network Architecture and Control Plane



This figure shows how Amazon Aurora separates its compute and storage layers while maintaining communication across distinct Virtual Private Clouds (VPCs).

Customer VPC

- Where the customer application resides.

The application connects to Aurora cluster endpoint for database operations—read and write requests are routed automatically to the correct database instance.

RDS VPC (Control Plane)

- RDS Host Manager (RDS HM) operates here.

It monitors the health of Aurora instances, manages failover, and handles communication between the database engine (Aurora MySQL) and the underlying storage.

This separation allows Aurora to recover quickly from failures without affecting user applications.

Primary and Secondary Databases

Primary RW DB (Read-Write): Handles all write operations.

Secondary RO DB (Read-Only): Handles read operations. In case of failure, takes over primary.

Same underlying storage volume => data consistency.

Storage VPC

- Distributed storage layer that forms the heart of Aurora.

It contains multiple Storage Nodes backed by local SSDs, which store replicated copies of database pages.

Backup and restore operations also occur here, often integrating with Amazon S3 for durability and long-term storage.

S3 Integration

Data and backups are regularly written to Amazon S3, this provides very high durability and simplifies disaster recovery.

If a storage node fails, data quickly reconstructed from other copies or S3 backups.

Workflow: Data Storage and Retrieval

How Data is Stored

Data is stored in organized structures inside storage systems (usually SSDs or cloud storage).

1. Tables and Pages:
    a. The data is stored in data pages
    b. Each page holds multiple rows of data.
2. Indexes:
    a. To find data faster, the database builds indexes (analogous to index of a book).
    b. Indexes are stored in a B-tree or similar structure so that the system can jump directly to the correct location instead of scanning the entire table.
3. Storage Nodes:
    a. Aurora doesn’t store data on the database server itself.
    b. Instead it is stored in six copies across three different Availability Zones (AZs).
    c. This means even if two copies or one entire zone fails, the data is safe.
4. Write-Ahead Log (WAL):

    a. Every change (insert, update, delete) is first written to a log before it's applied to the main data.
    b. This ensures durability - even if the system crashes => log replay and recover.

How Data is Retrieved (Read Operations)

Suppose the query is : SELECT * FROM students WHERE roll_no = 23;
Following happens:

1. Parsing and Optimization:
    a. The query is first parsed (Syntax error checking).
    b. Then, the query optimizer decides the best way to fetch data (e.g., use index or scan the full table?).
2. Execution:
    a. If an index exists on roll_no, it goes directly to the correct page.
    b. If not, then it scans the table sequentially.
3. Data Retrieval:
    a. The required pages are fetched from Aurora's distributed storage.
    b. If they're already in memory (cache), it reads them directly (very fast).
    c. If not, it pulls them from storage nodes (slower).
4. Result Sent to Client:
    a. Once the database gets the rows, it sends the result back.

How a Query is Handled Internally

1. Client → Cluster Endpoint:
   SQL query to Aurora cluster endpoint.
2. Primary Node Handles Writes:
    a. Aurora has one primary instance that handles write queries.
    b. Multiple replicas to handle read queries.
3. Coordinator (Query Processor):
    a. The coordinator parses the query, decides the best plan, and asks storage nodes for data.
4. Storage Layer:
    a. Aurora's storage layer is distributed and automatically managed.
    b. It fetches or writes data in parallel from multiple nodes for speed and fault tolerance.
5. Results are aggregated and then returned.

Concurrency Control

- Multi-Version Concurrency Control (MVCC) for non-blocking reads.
- Each transaction operates on a consistent snapshot of the database.
- Readers never block writers; writers serialize at commit time.

Logging & Recovery

- Uses redo log shipping rather than data pages.
- Continuous checkpoints are written asynchronously.
- Crash recovery only replays logs since last checkpoint → faster recovery.

REFERENCES:

1. Amazon Aurora: Analyzing the Architecture of a Cloud-Native Database | by Varshitha R | Medium
2. Verbitski, A., et al. "Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases"

How Amazon Aurora Handles Netflix's "Continue Watching" Feature

1. Scenario Overview

Netflix's Continue Watching feature tracks every user's playback position in real time => constant reads and writes to database. During major releases like Stranger Things, millions of users cause these actions simultaneously which creates a huge load. Aurora should be able to handle this.

2. Architectural Response

a. Write Operations

When a user pauses a show/movie :

update watch_history set position = 1260, last_updated = NOW() where user_id = 839245;

- The Aurora writer node logs only the change (redo records) to its distributed storage layer.
- Each data item is replicated six times across three Availability Zones (two copies per AZ).
- A write is confirmed after 4 of 6 replicas acknowledge (write quorum = 4/6).

b. Read Operations

When the same user resumes on another device or even the same device:

select title, position from watch_history where user_id = 839245;

- The query is routed to the nearest read replica.
- Aurora requires responses from only 3 of 6 replicas (read quorum = 3/6).
- Reads reflect the most recent writes instantly.
- Cached results in memory buffers reduce repeated fetch time to milliseconds.

Use Cases and Effectiveness

Ideal Scenarios

Cloud-native enterprise workloads needing high availability and low latency. E-commerce, financial transactions, analytics backends, and microservices that need strong consistency. SaaS platforms requiring multi-AZ fault tolerance.

Real-World Implementations

Amazon.com backend services, Netflix metadata service, Expedia and Airbnb workloads on AWS Aurora etc.

Strengths

- Fault-tolerant multi-AZ replication
- Fast recovery, independent of DB size
- Auto-scaling and repair
- Fully compatible with MySQL/PostgreSQL

Limitations / Challenges

- Aurora is proprietary to AWS; not portable.
- Write latency can increase under quorum loss conditions.
- Higher costs than self-managed MySQL.
- Limited access to low-level tuning parameters.

Contributor Details

Prepared By: Sreerag S
Date: November 2025